

Contents

BBC News — Module 3 Lab Guide	2
The Scenario — Your Mission	2
Where you work	2
The problem	3
Your manager's request	3
Your team's job for the next 2 weeks (Module 3)	3
After Module 3	3
Why this scenario is special	3
How This Guide Works (Read This First!)	4
We do NOT give you the full code	4
Encouraging words for nervous teams	4
Visualizations — Two Modes	4
1. Exploratory charts (for YOU)	4
2. Explanatory charts (for OLIVIA)	4
Your plotting toolkit (you learned this in Module 2 Class 5)	5
Before You Start — Google Colab Setup	5
Step 1 — Open a new Colab notebook	5
Step 2 — Connect to Google Drive	5
Step 3 — Create a project folder	5
Step 4 — Get the data	5
Step 5 — Install spaCy and download the English model	6
Step 6 — Test it	6
Colab tips	6
Class 1 — Data Cleaning	6
Your goal	7
Inputs	7
Outputs	7
Phase A — Explore the data first (15 minutes)	7
Phase B — Load and clean all articles (45 minutes)	7
Phase C — Make ONE chart for Olivia (15 minutes)	9
Common mistakes in Class 1	9
Self-check before Class 2	9
Class 2 — Encoding and Scaling	10
Your goal	10
Inputs	10
Outputs	10
Phase A — Explore the data first	10
Phase B — Encode the label and start the numeric features	11
Phase C — Make ONE chart for Olivia	12
Common mistakes in Class 2	12
Self-check before Class 3	12
Class 3 — Feature Engineering	13
Your goal	13
Inputs	13
Outputs	13
Phase A — Explore the data first	13
Phase B — Engineer the features	14
Phase C — Make ONE chart for Olivia	15
Common mistakes in Class 3	15

Self-check before Class 4	16
Class 4 — Feature Selection	16
Your goal	16
Inputs	16
Outputs	16
Phase A — Explore the data first	16
Phase B — Select features	17
Phase C — Make ONE chart for Olivia	18
Common mistakes in Class 4	18
Self-check before Class 5	18
Class 5 — Pipelines	18
Your goal	19
Inputs	19
Outputs	19
Phase A — Explore the data first	19
Phase B — Build the pipeline	19
Phase C — Make ONE chart for Olivia	20
Common mistakes in Class 5	21
Self-check before Class 6	21
Class 6 — End-to-End Lab (THE BIG ONE)	21
Your goal	21
Time	21
Inputs	21
Outputs	22
Phase A — Explore the data first (10 minutes)	22
Phase B — Build the final dataset (70 minutes)	22
Phase C — Findings report for Olivia (10 minutes)	23
Grading rubric (100 points)	23
Submit	24
Tips for the whole module	24

BBC News — Module 3 Lab Guide

Scenario: BBC newsroom. Auto-classify articles into the right desk (Business, Entertainment, Politics, Sport, Tech).
Module: 3 (Data Preparation and Feature Engineering) **Audience:** Adult learners, A2 English. New to AI. **Tools:** Google Colab (no install needed).

The Scenario — Your Mission

Where you work

You and your team are new data analysts at the **BBC** (British Broadcasting Corporation). The BBC is the biggest news organisation in the United Kingdom. Millions of people read BBC News every day.

Every day: - **10,000 articles** arrive from wire services (Reuters, AP, AFP). - Each article must go to the right desk: Business, Entertainment, Politics, Sport, or Tech. - An editor reads each title and sends the article to the correct desk.

The problem

A human editor reads the title, then routes the article. This is slow. Also, when the editor is tired (at 3 a.m.), mistakes happen. A sports story goes to the Politics desk. A tech story goes to Entertainment.

Bad routing means: - Slow publishing. The competition is faster. - Confused readers (a football story on the Business homepage). - Tired editors doing boring work instead of real journalism.

The Head of Editorial Workflow Tech is worried. She calls your team into a meeting on Monday morning.

Your manager's request

Your manager, **Olivia** (Head of Editorial Workflow Tech at BBC), tells you:

“Every day we get 10,000 articles from wire services. Each one needs to go to the right desk. Right now a human reads each title and routes it. It costs us money. It makes mistakes when they are tired.

Build me a model that reads an article and routes it in **0.1 seconds**.

Then we will do TWO things: 1. The article goes straight to the right desk. The editor only checks the difficult ones. 2. Our editors spend their time on real journalism, not on clicking ‘send to Sport’.

If the model is right 95% of the time, we save thousands of hours of editor work every year.”

Your team's job for the next 2 weeks (Module 3)

Olivia cannot do this alone. Her data is **2,225 articles in folders** (one folder per category). It is NOT a CSV. It is a ZIP of folders with .txt files inside.

Your job in Module 3: > **Turn 2,225 messy text files into ONE clean file. The clean file will be used to train the model in Module 4.**

The clean file is called `bbc_news_clean.parquet`. It must have **17 specific columns** (we will see them in Class 6).

After Module 3

Module	What happens
Module 4	Train the classification model. Olivia finally gets her auto-router.
Module 5	Find groups of articles (long-form vs short news). For homepage design.
Module 7	Read the full article text. Find what topics appear inside each category.

You use the **same BBC News dataset** until the end of Module 7.

Why this scenario is special

This is the **smallest** dataset in the course. Only **2,225 rows**. Most students will think “great, easy!” — and they are partly right. The data is small. The categories are nearly balanced (~445 articles each).

BUT — and this is important — the data is **TEXT**. There are almost no numeric columns. You must **CREATE the numeric columns from the text yourself**.

This is your first **NLP** (Natural Language Processing) project. You will learn how to count words, find names of people, find names of countries, and turn a paragraph into numbers a model can use.

No GPU needed. No deep learning. Just smart feature engineering.

How This Guide Works (Read This First!)

We do NOT give you the full code

In this guide you will see two kinds of content:

1. WHAT / WHY / EXPECTED — explained in words

For every step you read: - **WHAT** you have to do (in normal language) - **WHY** you do it (the reason) - **HINTS** — the function names, the arguments to remember - **EXPECTED** — the result you should see if you did it right

2. Sometimes a tiny code skeleton with blanks

For the harder steps we give you a code skeleton with ____ blanks. **You fill in the blanks.**

Why no full code?

Because you will NOT learn if you copy-paste. - In Module 4, the model only works if YOU understand each step. - In your future job, nobody will give you the code. - Writing the code yourself takes 10 minutes more today, but saves 10 hours later.

Rule: If you finish a step in 30 seconds by copy-pasting from a teammate's notebook, do it AGAIN by yourself. Different variable names, same logic.

Encouraging words for nervous teams

This is text data. You may have never worked with text before. **That is normal.**

The good news: - The dataset is **small** (2,225 rows). Loading is fast. Everything runs in seconds. - The categories are **clean** (5 clear labels). - The categories are **balanced** (about 445 articles each). - You do NOT need a GPU. You do NOT need to install heavy libraries. - You only need pandas, scikit-learn, and one new library: spaCy (for finding names in text).

If you finish early, help your teammates. Text data is new for everyone in the room.

Visualizations — Two Modes

In every class you will make charts. There are **two reasons** to make a chart:

1. Exploratory charts (for YOU)

Made BEFORE you clean the data. To understand what the data looks like. - Fast, ugly, no labels needed. - Examples: `df.hist()`, `df['column'].value_counts().plot.bar()`.

Goal: “**What does this data look like?**”

2. Explanatory charts (for OLIVIA)

Made AFTER you understand. To EXPLAIN something to your manager. - Clean, labeled, one clear message. - Must have: title, x-label, y-label, source, and a 1-sentence takeaway.

Goal: “**Olivia, look at this. This is the pattern.**”

In every class you make BOTH kinds.

Your plotting toolkit (you learned this in Module 2 Class 5)

Plot	When to use it	Function name
Histogram	See the SHAPE of a numeric column	<code>plt.hist()</code> or <code>sns.histplot()</code>
Bar chart	Count of each category	<code>df['col'].value_counts().plot.bar()</code>
Box plot	See outliers in a numeric column	<code>sns.boxplot()</code>
Scatter plot	See the relationship between 2 numbers	<code>plt.scatter()</code> or <code>sns.scatterplot()</code>
Heatmap	See correlation between many columns	<code>sns.heatmap(df.corr())</code>
Word cloud	See most common words in a text column	<code>WordCloud().generate(text)</code>

Before you start: `import matplotlib.pyplot as plt` and `import seaborn as sns`.

Before You Start — Google Colab Setup

You will work in **Google Colab**. That means: - You do NOT install Python. - You do NOT install pandas, scikit-learn, or anything (almost). - You just open a notebook in your web browser.

Step 1 — Open a new Colab notebook

1. Go to <https://colab.research.google.com>
2. Sign in with your Google account.
3. Click “**New notebook**” (top left).
4. Name it `bbc_news_module_3.ipynb`.

Step 2 — Connect to Google Drive

Colab files **disappear** when you close the browser. So you must save your work to **Google Drive**.

In the first cell:

```
from google.colab import drive
drive.mount('/content/drive')
```

A pop-up asks for permission. Click “Allow”. After this, Drive is available at `/content/drive/MyDrive/`.

Step 3 — Create a project folder

In a new cell:

```
import os
os.makedirs('/content/drive/MyDrive/bbc_news_lab', exist_ok=True)
%cd /content/drive/MyDrive/bbc_news_lab
```

Your team will save ALL files in this folder.

Step 4 — Get the data

The data is at: <http://mlg.ucd.ie/datasets/bbc.html> (the original source) or on Kaggle as “BBC News Classification”.

Option A — Direct download (recommended):

In Colab:

```
!wget http://mlg.ucd.ie/files/datasets/bbc-fulltext.zip
!unzip -q bbc-fulltext.zip
!ls bbc/
```

You should see **5 folders**: business, entertainment, politics, sport, tech. Each folder has many .txt files.

Option B — From Kaggle:

1. Make a free Kaggle account.
2. Go to kaggle.com/settings/account, scroll to “API”, click “Create New Token”. Save the `kaggle.json` file.
3. In Colab:

```
from google.colab import files
files.upload() # pick kaggle.json
```

4. Then:

```
!mkdir -p ~/.kaggle && mv kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json
!pip install kaggle -q
!kaggle datasets download -d pariza/bbc-news-summary
!unzip -q bbc-news-summary.zip
```

Step 5 — Install spaCy and download the English model

We need one new library: **spaCy**. It finds names of people, places, and companies in text.

```
!pip install spacy -q
!python -m spacy download en_core_web_sm
```

This takes about 30 seconds.

Step 6 — Test it

```
import os
folders = os.listdir('bbc/')
print(folders)
```

Should print something like `['business', 'entertainment', 'politics', 'sport', 'tech']`. You are ready.

Colab tips

Tip	Why
Save your notebook to Drive often	Colab disconnects after 12 hours.
Save intermediate .parquet files to Drive	Files in /content/ disappear when Colab disconnects.
Share the notebook with teammates (Share button, top right)	Editor access. Like Google Docs for code.
Restart runtime if memory full	Runtime > Restart runtime.

Class 1 — Data Cleaning

Scenario reminder: Olivia gives you a ZIP file with 5 folders inside. Each folder has about 445 .txt files. The folder name is the category. The file content is the article. Your job today: load all 2,225 files into ONE DataFrame and clean the text.

Your goal

Read 2,225 .txt files into one DataFrame. Make 4 columns: article_id, category, headline, text. Clean strange characters.

Inputs

- The 5 folders in bbc/

Outputs

- articles_step1.parquet saved in your Drive folder
 - 3+ exploratory charts in your notebook
 - 1 explanatory chart for Olivia
 - Notes in markdown about every decision you made
-

Phase A — Explore the data first (15 minutes)

Before you load anything, **LOOK** at one file by hand.

Exploratory chart 1 — How many files per category?

- **Question:** “Are the 5 categories balanced? Or is one much bigger?”
- **HINTS:**
 - Use `os.listdir('bbc/<folder>/')` for each of the 5 folders.
 - Use `len(...)` on the result.
 - Plot the 5 counts as a bar chart.
- **What you learn:** The categories should be roughly equal (around 400-510 articles each).

Exploratory chart 2 — Open ONE file by hand

- **Question:** “What does an article look like? What is the headline? Where does the body start?”
- **HINTS:**
 - Pick any file. Example: `bbc/sport/001.txt`.
 - Open with `open(path, 'r', encoding='latin-1').read()`.
 - Print the first 500 characters.
- **What you learn:** The first line is the HEADLINE. Then a blank line. Then the body.

Exploratory chart 3 — Article length distribution (rough estimate)

- **Question:** “How long is a typical BBC article?”
 - **HINTS:**
 - Pick 100 random files. Count characters of each.
 - Histogram the lengths.
 - **What you learn:** Articles range from ~500 to ~5000 characters. Most are around 2000.
-

Phase B — Load and clean all articles (45 minutes)

Step 1 — Walk through all 5 folders

- **WHAT:** Visit each of the 5 folders. List the files. Read each file. Build a list of dictionaries.
- **HINTS:**
 - Use `os.listdir('bbc/<category>/')` to get all filenames.

- Loop over the 5 categories.
- For each file, build a dict: {'article_id': ..., 'category': ..., 'raw_text': ...}.
- Append to a list.
- **WARNING about encoding:** Some files have non-ASCII characters (special quotes, accents). Use `encoding='latin-1'` when opening, not `'utf-8'`. Some files crash with `utf-8`.
- **EXPECTED:** A list of 2,225 dicts.

Step 2 — Convert the list to a DataFrame

- **WHAT:** Turn the list of dicts into a pandas DataFrame.
- **HINTS:**
 - `df = pd.DataFrame(your_list)`.
- **EXPECTED:** A DataFrame with 2,225 rows and 3 columns: `article_id`, `category`, `raw_text`.

Step 3 — Split each article into headline + body

- **WHAT:** The first line of each article is the HEADLINE. The rest is the body.
- **HINTS:**
 - Use `.str.split('\n', n=1)` on the `raw_text` column. The argument `n=1` means “split only once”.
 - The result is a list of 2 strings: [headline, body].
 - Save them as 2 new columns: `headline`, `body`.
- **WHY:** The headline is very useful for prediction. A title like “Manchester United beat Arsenal” is obviously SPORT. The first 3 words alone often give the answer.
- **EXPECTED:** Two new columns. Print 5 random headlines to confirm.

Step 4 — Strip whitespace and remove empty lines from the body

- **WHAT:** The body has many `\n\n` (double newlines, between paragraphs). Some have leading or trailing spaces. Clean them up.
- **HINTS:**
 - Use `.str.strip()` to remove leading/trailing spaces.
 - To keep paragraph breaks but remove extra blank lines: split by `'\n\n'`, filter empty strings, join with `'\n\n'`. You can do this in a small function and `.apply()` it.
- **WHY:** Clean text gives better word counts.

Step 5 — Find missing or empty articles

- **WHAT:** Count rows where body is empty or very short (< 100 characters).
- **HINTS:**
 - `(df['body'].str.len() < 100).sum()` counts them.
- **EXPECTED:** Probably 0 or very few. If > 0, decide: drop or keep.

Step 6 — Check encoding problems (strange characters)

- **WHAT:** Look for weird characters like `Â£` (this is a broken £ sign) or other broken non-ASCII.
- **HINTS:**
 - Print a few articles. Search for `Â`, `â`, `€`, `Ã`.
 - If found, you can use `str.replace()` to fix them. Common fixes:
 - * `Â£` → `£`
 - * `â€™` → `'` (a smart quote)
 - * `â€œ` → `"` (an opening smart quote)
- **WHY:** Broken characters confuse the word counter and the named-entity recogniser.

Step 7 — Write down what you did

In a markdown cell, write: - Starting articles: 2,225 - After dropping empty: ____ - Encoding fixes applied: ____ - WHY for each decision.

Step 8 — Save to Drive

- **WHAT:** Save the cleaned DataFrame as parquet, inside your Drive folder.
 - **HINTS:** `df.to_parquet('/content/drive/MyDrive/bbc_news_lab/articles_step1.parquet')`.
-

Phase C — Make ONE chart for Olivia (15 minutes)

She does not have time to read your code. She wants ONE picture.

Olivia's chart — “Are the 5 categories balanced?”

Make a bar chart showing the count of articles per category.

Category	Count
Business	~510
Entertainment	~386
Politics	~417
Sport	~511
Tech	~401

- **HINTS:**
 - Use `df['category'].value_counts().plot.bar()`.
 - Add the title: "BBC News dataset – 2,225 articles in 5 nearly-balanced categories".
 - X-label: category name.
 - Y-label: number of articles.
 - Put the number on top of each bar.
 - **Takeaway for Olivia:** “The 5 categories are almost equal. The model will not be biased toward one class.”
-

Common mistakes in Class 1

Mistake	What goes wrong
Use <code>encoding='utf-8'</code> when reading files	Crash on some files with non-ASCII characters.
Forget to keep category (the folder name)	You lose the target column.
Split headline / body with <code>n=2</code> instead of <code>n=1</code>	You lose part of the body.
Save text columns as CSV	Quotes and newlines break the CSV. Use parquet.

Self-check before Class 2

- ☐ You have a DataFrame with 2,225 rows.
 - ☐ Four columns: `article_id`, `category`, `headline`, `body`.
 - ☐ No empty bodies.
 - ☐ No obvious broken characters.
 - ☐ At least 3 exploratory charts in your notebook.
 - ☐ 1 polished chart for Olivia.
 - ☐ You saved `articles_step1.parquet` to your Drive folder.
 - ☐ You wrote down (in markdown) what you did and WHY.
-

Class 2 — Encoding and Scaling

Scenario reminder: Olivia looks at your DataFrame. She is happy. But she says: “the model is a math model. It does not understand the word ‘Politics’. It does not understand the article text either. Turn it all into numbers.”

Your goal

Turn the TEXT into numbers. Turn the CATEGORY label into numbers. Make the first basic numeric features.

Inputs

- `articles_step1.parquet` from Class 1

Outputs

- `articles_step2.parquet` in Drive
 - 3+ exploratory charts
 - 1 explanatory chart for Olivia
-

Phase A — Explore the data first

Exploratory chart 1 — Article length per category

- **Question:** “Are Sport articles shorter than Business articles? Or longer?”
- **HINTS:**
 - First make a column `article_length = df['body'].str.len()`.
 - Use `sns.boxplot(x='category', y='article_length', data=df)`.
- **What you learn:** Tech and Business articles tend to be longer. Sport is shorter.

Exploratory chart 2 — Headline length distribution

- **Question:** “Are headlines short (5 words) or long (15 words)?”
- **HINTS:**
 - `headline_word_count = df['headline'].str.split().str.len()`.
 - Histogram with `bins=20`.
- **What you learn:** BBC headlines are usually 5-10 words.

Exploratory chart 3 — Most common words in each category (word cloud)

- **Question:** “What words appear most in Sport vs Tech?”
- **HINTS:**
 - `!pip install wordcloud -q`.
 - `from wordcloud import WordCloud`.
 - For each category, join all articles into one big string.
 - `WordCloud().generate(big_text).to_image()`.
- **What you learn:** Sport = “team, game, won”. Tech = “computer, software, internet”. This is your gut check that the categories are real.

Exploratory chart 4 — Average article length per category

- **Question:** Bar chart showing mean character length per category.
- **HINTS:**
 - `df.groupby('category')['article_length'].mean().plot.bar()`.

- **What you learn:** Confirms what the boxplot showed.
-

Phase B — Encode the label and start the numeric features

Step 1 — Encode the category column to numbers

- **WHAT:** Turn the 5 text labels into 5 numbers (0, 1, 2, 3, 4).
- **HINTS:**
 - Use `from sklearn.preprocessing import LabelEncoder`.
 - `le = LabelEncoder()`.
 - `df['category_id'] = le.fit_transform(df['category'])`.
 - Save the mapping. Print `dict(zip(le.classes_, range(5)))` to see which number means which category.
- **WHY:** The model in Module 4 needs numbers, not words.
- **WARNING:** Keep BOTH columns: `category` (text, for charts) and `category_id` (number, for the model).

Step 2 — Compute `article_length` (in characters)

- **HINTS:** `df['article_length'] = df['body'].str.len()`.

Step 3 — Compute `n_words`

- **WHAT:** Number of words in the body.
- **HINTS:** `df['n_words'] = df['body'].str.split().str.len()`.

Step 4 — Compute `n_sentences`

- **WHAT:** A sentence ends with `.`, `!`, or `?`. Count them.
- **HINTS:**
 - Use a regex with `.str.count(...)`. Skeleton:
`df['n_sentences'] = df['body'].str.count(r'____')` *# YOU fill in: pattern for . ! or ?*
 - The pattern is `r'[.!?]'`.
- **WARNING:** This is not perfect (abbreviations like “Mr.” also have a dot). But for BBC news it works well enough.

Step 5 — Compute `n_paragraphs`

- **WHAT:** Paragraphs are separated by `\n\n` (double newline).
- **HINTS:**
 - `df['n_paragraphs'] = df['body'].str.count(r'\n\n') + 1`.
- **WHY +1?** `N` separators make `N+1` paragraphs.

Step 6 — Compute `avg_word_length`

- **WHAT:** The average length (in letters) of words in the article.
- **HINTS:**
 - Define a small function: take a string, split into words, take `len()` of each, return mean.
 - Apply it with `df['body'].apply(your_function)`.
- **WHY:** Tech articles use longer words (“infrastructure”, “encryption”). Sport uses shorter words (“won”, “goal”, “team”).

Step 7 — Compute `n_unique_words` and `lexical_diversity`

- **WHAT:**
 - `n_unique_words` = number of different words used.
 - `lexical_diversity` = `n_unique_words / n_words` (between 0 and 1).

- **HINTS:**
 - In a function: `split()`, `lower-case`, put in `set()`, take `len()`.
- **WHY:** Politics articles repeat the same words (“government”, “minister”). Lexical diversity is LOWER.

Step 8 — Scale numeric columns (preview only)

- **WHAT:** Use `StandardScaler` to make every numeric column have mean = 0, std = 1.
- **HINTS:**
 - `from sklearn.preprocessing import StandardScaler.`
 - `scaler = StandardScaler().`
 - `scaler.fit_transform(df[numeric_cols]).`
- **WARNING:** This is for inspection only. In Class 5, scaling goes INSIDE the Pipeline. Do NOT save the scaled version as your final file.

Step 9 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/bbc_news_lab/articles_step2.parquet').`

Phase C — Make ONE chart for Olivia

Olivia’s chart — “Article length depends on the desk”

A box plot of article length (in words) per category.

- **HINTS:**
 - `sns.boxplot(x='category', y='n_words', data=df).`
- **Title:** “Article length by category — Tech and Business write the longest, Sport writes the shortest”.
- **X-label:** Category.
- **Y-label:** Number of words per article.
- **Takeaway for Olivia:** “Just the article length already tells us something about the category. The Sport desk produces shorter, faster news. Tech and Business need more space to explain.”

Common mistakes in Class 2

Mistake	What goes wrong
Scale BEFORE train/test split	Leakage. Scaler sees test data.
Drop the category text column too early	You cannot make pretty charts after that.
Forget to save <code>LabelEncoder</code> mapping	You cannot convert predictions back to category names later.
Save the scaled version	Class 5 will scale again inside the pipeline. Double scaling = wrong numbers.

Self-check before Class 3

- ☐ `category_id` column exists (5 numbers).
- ☐ You wrote down which number means which category.
- ☐ `article_length`, `n_words`, `n_sentences`, `n_paragraphs` exist.
- ☐ `avg_word_length`, `n_unique_words`, `lexical_diversity` exist.
- ☐ At least 3 exploratory charts.
- ☐ 1 chart for Olivia.
- ☐ `articles_step2.parquet` saved to Drive.

Class 3 — Feature Engineering

Scenario reminder: Olivia says: “The simple counts you made yesterday are nice. But the TRULY useful columns are not there. For example: how many names of countries are in the article? How many companies? How many people? A Politics article is full of country names. A Sport article is full of player names. Find them.”

Your goal

Make NEW columns that capture WHAT is in the article, not just HOW LONG it is. This is where the model gets its real power. **Named-entity recognition (NER)** is the key tool today.

Inputs

- `articles_step2.parquet`

Outputs

- `articles_step3.parquet` in Drive
 - 3+ exploratory charts
 - 1 explanatory chart for Olivia
-

Phase A — Explore the data first

Exploratory chart 1 — Run spaCy on ONE article

- **Question:** “What does spaCy find in a Politics article? In a Sport article?”
- **HINTS:**
 - `import spacy.`
 - `nlp = spacy.load('en_core_web_sm').`
 - Pick one Politics article. Run `doc = nlp(text).`
 - Loop over `doc.ents`. Print `ent.text`, `ent.label_` for each.
- **What you learn:** spaCy labels each name. PERSON (a human), GPE (a country or city), ORG (a company or party). You will count these.

Exploratory chart 2 — Named-entity types across categories

- After Step 2-4 below, you will have counts of PERSON, GPE, ORG per article.
- **Question:** “Do Politics articles have more GPE (countries) than Sport?”
- **HINTS:** `df.groupby('category')[['n_named_entities_persons', 'n_named_entities_locations', 'n_named_entities_organizations']].mean().plot.bar().`
- **What you learn:** Politics = lots of locations. Sport = lots of persons. Business = lots of organisations.

Exploratory chart 3 — Number of CAPS words per category

- **Question:** “Do Tech articles use more ALL-CAPS words (acronyms like IBM, NASA)?”
- **HINTS:**
 - After Step 5, look at `df.groupby('category')['n_caps_words'].mean().`
 - Bar chart.
- **What you learn:** Tech and Business use more acronyms.

Exploratory chart 4 — has_quote rate per category

- **Question:** “Which desks quote people the most?”
 - **HINTS:** `df.groupby('category')['has_quote'].mean().plot.bar()`.
 - **What you learn:** Politics and Sport often quote people (politicians, players). Tech less so.
-

Phase B — Engineer the features

Step 1 — Set up spaCy ONCE (it is slow to load)

- **WHAT:** Load the spaCy model once, save in a variable.
- **HINTS:**
 - `import spacy.`
 - `nlp = spacy.load('en_core_web_sm')`.
- **WARNING:** Do NOT load it inside a loop. It would take 20 minutes. Load once, reuse 2,225 times.

Step 2 — Run spaCy on every article

This is the **biggest** step. It will take 2-5 minutes for 2,225 articles. That is fine.

- **WHAT:** For each article, run `nlp(text)`. Store the entities.
- **HINTS:**
 - For speed, use `nlp.pipe(df['body'], batch_size=50)` — this is FASTER than calling `nlp(text)` one by one.
 - For each doc, collect the entities into a list of `(text, label)` tuples.
 - Save the list in a new column `entities` (yes, a column of lists is allowed in pandas).
- **EXPECTED:** Each row has a list like `[('Tony Blair', 'PERSON'), ('UK', 'GPE'), ('Labour Party', 'ORG'), ...]`.

Step 3 — Count named entities by type

Make 3 new columns from the `entities` list:

New column	What it is
<code>n_named_entities_persons</code>	Count of entities where <code>label = PERSON</code>
<code>n_named_entities_locations</code>	Count of entities where <code>label = GPE</code> or <code>LOC</code>
<code>n_named_entities_organizations</code>	Count of entities where <code>label = ORG</code>

- **HINTS:**
 - Write a small function that takes the list and a target label. Returns the count.
 - Apply it with `df['entities'].apply(lambda lst: sum(1 for t, lab in lst if lab == 'PERSON'))`.

Step 4 — Drop the raw entities column (or keep for Module 7)

- **WHAT:** The 3 count columns are what the model needs. The raw `entities` list is heavy.
- **OPTIONS:**
 - **A:** Drop it. Save space.
 - **B:** Keep it for Module 7. Heavier file but useful later.
- **RECOMMENDED:** Keep it for now. Drop it in Class 6 if needed.

Step 5 — Count CAPS words

- **WHAT:** Count words that are ALL UPPERCASE and have at least 2 letters (so we ignore “I” and “A”).
- **HINTS:**

- In a function: `split`, filter `w.isupper()` and `len(w) >= 2`, count.
- Apply.
- **WHY:** Acronyms (IBM, NASA, BBC, CEO) are common in Tech and Business.

Step 6 — has_quote flag

- **WHAT:** Does the article contain a " character? (Or smart quotes if you fixed them in Class 1.)
- **HINTS:**
 - `df['has_quote'] = df['body'].str.contains('"').astype(int)`.
- **WHY:** Quoting someone is a Politics / Sport pattern.

Step 7 — Headline features

New column	What it is
<code>first_word_length</code>	Length (letters) of the first word in the HEADLINE
<code>headline_first_3_words</code>	The first 3 words of the headline, joined as one string
<code>headline_word_count</code>	How many words in the headline

- **HINTS:**
 - Split the headline. Take the first word. `len()` it.
 - For `headline_first_3_words`: `' '.join(words[:3])`.
- **WHY:** “Real Madrid beat Barcelona” → `headline_first_3_words = "Real Madrid beat"`. The model can learn that “Real Madrid” almost always means SPORT.

Step 8 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/bbc_news_lab/articles_step3.parquet')`.

Phase C — Make ONE chart for Olivia

Olivia’s chart — “Named entities tell the story of the desk”

A grouped bar chart: for each category, show 3 bars: average PERSON count, average LOCATION count, average ORG count.

- **HINTS:**
 - `df.groupby('category')[['n_named_entities_persons', 'n_named_entities_locations', 'n_named_entities_organizations']].mean().plot.bar()`.
- **Title:** “Named entities by desk — Sport = persons, Politics = locations, Business = organisations”.
- **X-label:** Category.
- **Y-label:** Average count per article.
- **Takeaway for Olivia:** “Even without reading a single word, we can tell the desk by counting names. Sport mentions players. Politics mentions countries. Business mentions companies. The model will love these features.”

Common mistakes in Class 3

Mistake	What goes wrong
Load <code>spacy.load()</code> inside the loop	20 minutes instead of 3. Load once.
Process one article at a time instead of <code>nlp.pipe()</code>	4x slower.
Forget to check spaCy label names (PERSON vs Person)	Counts are all zero.
Use GPE only, ignore LOC	Some locations get missed.

Self-check before Class 4

- ☐ entities column made (or counts only).
 - ☐ n_named_entities_persons, n_named_entities_locations, n_named_entities_organizations exist.
 - ☐ n_caps_words exists.
 - ☐ has_quote exists (0 or 1).
 - ☐ first_word_length, headline_first_3_words exist.
 - ☐ 3+ exploratory charts.
 - ☐ 1 explanatory chart.
 - ☐ articles_step3.parquet saved to Drive.
-

Class 4 — Feature Selection

Scenario reminder: Now you have ~14 engineered columns. Olivia says: “Some of these say the same thing. n_words and article_length are basically twins. Pick the ones that really help. I want a small, clean set.”

Your goal

Pick the best columns. Drop the rest. Justify every choice.

Inputs

- articles_step3.parquet

Outputs

- articles_step4.parquet in Drive (only the selected columns + category_id)
 - 3+ exploratory charts (correlation heatmap, mutual info bars, importance bars)
 - A short markdown report explaining your choices
-

Phase A — Explore the data first

Exploratory chart 1 — Correlation heatmap of numeric columns

- **Question:** “Which columns say the same thing as each other?”
- **HINTS:**
 - Pick only the numeric columns (skip category, headline, body, entities, headline_first_3_words).
 - Compute `df[numeric_cols].corr()`.
 - Plot with `sns.heatmap()`.
 - Use `annot=True` and `fmt='.2f'`.
- **What you learn:** n_words, article_length, n_sentences are highly correlated. You only need ONE of them.

Exploratory chart 2 — Mutual information bar chart

- After you compute mutual info (Step 4 below), plot it.
- **HINTS:**
 - Sort the values, then bar chart.
- **What you learn:** Which columns predict category_id strongest.

Exploratory chart 3 — Random Forest feature importance

- After you train the RF (Step 5 below), plot the importances.
 - **HINTS:** `pd.Series(rf.feature_importances_, index=...).sort_values().plot.barh()`.
 - **What you learn:** A second opinion on feature importance.
-

Phase B — Select features

Step 1 — Split into train and test FIRST

- **WHAT:** Use `train_test_split`.
- **HINTS:**
 - `from sklearn.model_selection import train_test_split.`
 - `X = df.drop(['category', 'category_id', 'headline', 'body', 'entities', 'headline_first_3_words'], axis=1).`
 - `y = df['category_id'].`
 - Pass: `test_size=0.2, random_state=42, stratify=y.`
- **WHY stratify=y?** The 5 categories are balanced. We want each split to also be balanced.
- **WHY:** All next steps use TRAIN ONLY. Otherwise leakage.

Step 2 — Remove low-variance columns

- **WHAT:** Drop columns where almost all values are the same.
- **HINTS:**
 - `from sklearn.feature_selection import VarianceThreshold.`
 - `VarianceThreshold(threshold=0.01).fit_transform(X_train).`
- **EXPECTED:** Probably none of yours will be dropped. Most have real variance.

Step 3 — Remove highly correlated columns

- **WHAT:** Drop one of each pair where `lccorr > 0.9`.
- **HINTS:**
 - Compute correlation matrix.
 - Get the upper triangle with `np.triu(...)`.
 - For each column, if any of its upper-triangle correlations is `> 0.9`, mark it for dropping.
- **EXPECTED:** `n_words` and `article_length` are very correlated. Keep one.

Step 4 — Rank by mutual information

- **WHAT:** Score each column by how much it tells you about `category_id`.
- **HINTS:**
 - `from sklearn.feature_selection import mutual_info_classif.`
 - `mi = mutual_info_classif(X_train_numeric, y_train).`
 - Put in a Series, sort.
- **EXPECTED:** `n_named_entities_persons`, `n_named_entities_locations`, `n_named_entities_organizations`, `avg_word_length`, `lexical_diversity` should be at the top.

Step 5 — Random Forest importance (second opinion)

- **WHAT:** Train a small RF, look at `feature_importances_`.
- **HINTS:**
 - `from sklearn.ensemble import RandomForestClassifier.`
 - `RandomForestClassifier(n_estimators=100, max_depth=8, n_jobs=-1, random_state=42).`
 - `.fit(X_train, y_train).`
 - Look at `.feature_importances_`.

Step 6 — Pick the final columns

- **WHAT:** Combine the rankings. Pick columns that:
 - Are high in mutual info, AND
 - Are high in RF importance, AND
 - Were not dropped by variance/correlation pruning.
- **WRITE DOWN:** In your notebook, list the columns. Explain in 1 sentence why each one is in.

Step 7 — Save

- **HINTS:** Keep the selected columns + `category_id` + `category` (for charts) + `body` (for Module 7) + `headline`. Save as `articles_step4.parquet` to Drive.
-

Phase C — Make ONE chart for Olivia

Olivia's chart — “These are the most important features”

A horizontal bar chart of your top features and their importance score.

- **HINTS:**
 - Use the RF importance.
 - `sort_values()` then `.plot.barh()`.
 - **Title:** “Top features for routing articles”.
 - **Y-axis:** column names.
 - **X-axis:** Random Forest importance.
 - **Takeaway:** “Three named-entity counts and the average word length predict most of the routing. The rest are small helpers.”
-

Common mistakes in Class 4

Mistake	What goes wrong
Compute mutual info on the FULL data	Leakage.
Drop the <code>body</code> and <code>headline</code> columns now	You need them for Module 7. Keep them.
Keep too many columns (20+) for a 2,225-row dataset	Overfitting risk.
Drop a high-mutual-info column	Big mistake. Check why it is high before dropping.

Self-check before Class 5

- ☐ Train/test split done FIRST.
 - ☐ Final feature set chosen (we recommend 8-12 columns).
 - ☐ `body` and `headline` kept (for Module 7).
 - ☐ You wrote down WHY for each kept column.
 - ☐ 3+ exploratory charts.
 - ☐ 1 chart for Olivia.
 - ☐ `articles_step4.parquet` saved to Drive.
-

Class 5 — Pipelines

Scenario reminder: Olivia says: “Your cleaning code is in 4 different notebooks. When a new article arrives tomorrow morning at 6 a.m., you cannot run 4 notebooks. We need ONE object that does everything.”

Your goal

Put EVERY cleaning step inside ONE Pipeline object. Production-ready code.

Inputs

- `articles_step4.parquet` (selected columns)

Outputs

- `bbc_pipeline.joblib` saved in Drive
 - 1 confusion-matrix chart + 1 classification report
 - A pipeline that takes engineered features and produces predictions
-

Phase A — Explore the data first

Exploratory chart 1 — Confusion matrix (after training)

- After Step 6, build a confusion matrix.
- **HINTS:**
 - `from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay.`
 - `cm = confusion_matrix(y_test, y_pred).`
 - `ConfusionMatrixDisplay(cm, display_labels=['business', 'entertainment', 'politics', 'sport', 'tech']).plot().`
- **What you learn:** How many articles per category did we get right? Where are the mistakes? (Common mistake: Politics → Business.)

Exploratory chart 2 — Classification report (table)

- **HINTS:**
 - `from sklearn.metrics import classification_report.`
 - `print(classification_report(y_test, y_pred, target_names=['business', ...])).`
 - **What you learn:** Precision and recall PER category. Which desk is the hardest to predict?
-

Phase B — Build the pipeline

Step 1 — Decide which columns are numeric and which are categorical

- **HINTS:** Make two lists.
- **EXAMPLE:**
 - numeric: `article_length, n_words, n_sentences, n_paragraphs, avg_word_length, lexical_diversity, n_named_entities_persons, n_named_entities_locations, n_named_entities_organizations, n_caps_words, first_word_length, headline_word_count.`
 - categorical: `has_quote` (already 0/1, so optional).

Step 2 — Build the numeric mini-pipeline

- **WHAT:** 2 steps: imputer (fill missing) + scaler.
- **HINTS:**
 - `from sklearn.pipeline import Pipeline.`
 - `from sklearn.impute import SimpleImputer.`
 - `from sklearn.preprocessing import StandardScaler.`
 - Skeleton:

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('scaler', StandardScaler()),
])
```

- Fill in the blank: best strategy for numeric? ('median')

Step 3 — Build the categorical mini-pipeline (only if you have any)

- **WHAT:** 2 steps: imputer + one-hot encoder.
- **HINTS:**
 - from sklearn.preprocessing import OneHotEncoder.
 - Skeleton:

```
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('onehot', OneHotEncoder(handle_unknown='___', sparse_output=___)),
])
```
 - Fill in blanks. (Strategy 'most_frequent', handle_unknown 'ignore', sparse_output False.)
- **WHY handle_unknown='ignore'?** In production, a new unseen value may appear. We do not want the code to crash.

Step 4 — Combine into a ColumnTransformer

- **HINTS:**
 - from sklearn.compose import ColumnTransformer.
 - It takes a list of tuples: (name, transformer, list_of_columns).

Step 5 — Add the model on top

- **HINTS:**
 - from sklearn.linear_model import LogisticRegression.
 - LogisticRegression(max_iter=1000, random_state=42, multi_class='auto').
 - Put it in a Pipeline with the preprocessor.
- **WHY no class_weight='balanced'?** Our 5 categories are already nearly balanced (~445 each). We do not need it.

Step 6 — Train and evaluate

- **HINTS:**
 - full_pipeline.fit(X_train, y_train).
 - y_pred = full_pipeline.predict(X_test).
 - from sklearn.metrics import accuracy_score, classification_report.
- **EXPECTED:** Accuracy around 0.80-0.90 with just these engineered features. (Module 4 will push this to 95%+ with TF-IDF on the body text.)

Step 7 — Save the trained pipeline

- **HINTS:**
 - import joblib.
 - joblib.dump(full_pipeline, '/content/drive/MyDrive/bbc_news_lab/bbc_pipeline.joblib').

Phase C — Make ONE chart for Olivia

Olivia's chart — “How many articles do we route correctly?”

A confusion matrix as a heatmap.

	Predicted Business	Predicted Entertainment	Predicted Politics	Predicted Sport	Predicted Tech
Actually Business	high	low	low	low	low
Actually Entertainment	low	high	low	low	low
Actually Politics	some	low	high	low	low
Actually Sport	low	low	low	high	low
Actually Tech	low	low	low	low	high

- **HINTS:** Use the `ConfusionMatrixDisplay` from Phase A. Add a title.
- **Title:** “Of 445 articles per desk, our baseline model routes 80-90% to the right place”.
- **Takeaway for Olivia:** “We are not at 95% yet. Module 4 will add the TF-IDF features (counting EVERY word) and we will get there. But this proves the pipeline works end-to-end.”

Common mistakes in Class 5

Mistake	What goes wrong
Build the pipeline AFTER you scaled the data manually	Pipeline scales it twice. Numbers wrong.
Forget <code>sparse_output=False</code> in <code>OneHotEncoder</code>	Some downstream code breaks.
Pass the text category column instead of <code>category_id</code>	Some models do not accept string labels.
Forget <code>random_state</code>	Results change every run. Hard to debug.

Self-check before Class 6

- ☐ Pipeline has preprocessor + model.
- ☐ Preprocessor has numeric (and maybe categorical) transformers.
- ☐ `handle_unknown='ignore'` set if used.
- ☐ Confusion matrix + classification report charts.
- ☐ `bbc_pipeline.joblib` saved to Drive.

Class 6 — End-to-End Lab (THE BIG ONE)

Scenario reminder: Olivia is in the meeting room. She wants the FINAL clean dataset on her desk in 90 minutes. This is the lab.

Your goal

Take 5 folders of `.txt` files (2,225 articles). Produce ONE final `.parquet` file with the exact 17-column schema. Plus a 1-page findings report.

Time

90 minutes of focused work.

Inputs

- The 5 folders of `.txt` files in your Drive folder

Outputs

- `bbc_news_clean.parquet` (~2,200 rows × 17 columns) in Drive
- `findings.md` (~1 page)
- Your full Colab notebook

Phase A — Explore the data first (10 minutes)

You will reuse charts from Classes 1-5. Pick 3 of them. Put them all on ONE PAGE of your notebook with markdown headers: 1. Article counts per category (the “5 nearly-balanced classes” chart) 2. Named entities by category (your most important chart) 3. Top features by Random Forest importance

These 3 charts tell Olivia the whole story.

Phase B — Build the final dataset (70 minutes)

Required output schema

Your `bbc_news_clean.parquet` MUST have these 17 columns, with these names and types:

#	Column	Type	Source
1	<code>article_id</code>	string	engineered (folder + filename)
2	<code>category</code>	string	TARGET (folder name)
3	<code>article_length</code>	int	engineered (characters in body)
4	<code>n_sentences</code>	int	engineered (count of . ! ?)
5	<code>n_paragraphs</code>	int	engineered (count of double newlines + 1)
6	<code>avg_word_length</code>	float	engineered (mean letters per word)
7	<code>n_unique_words</code>	int	engineered (size of word set)
8	<code>lexical_diversity</code>	float	engineered (n_unique / n_words)
9	<code>n_named_entities_persons</code>	int	engineered with spaCy (PERSON)
10	<code>n_named_entities_locations</code>	int	engineered with spaCy (GPE + LOC)
11	<code>n_named_entities_organizations</code>	int	engineered with spaCy (ORG)
12	<code>n_caps_words</code>	int	engineered (ALL-CAPS words, length >= 2)
13	<code>has_quote</code>	int (0/1)	engineered (contains “)
14	<code>first_word_length</code>	int	engineered (length of first headline word)
15	<code>headline_first_3_words</code>	string	engineered (first 3 words of headline)
16	<code>headline</code>	string	engineered (first line of file)
17	<code>body</code>	string	engineered (rest of file — kept for Module 7!)

Lab phases (timed)

Phase	Time	What you do
1. Load	10 min	Walk through 5 folders. Read all 2,225 .txt files. Build DataFrame.
2. Clean	10 min	Split headline / body. Strip whitespace. Fix encoding issues.
3. Basic features	15 min	article_length, n_sentences, n_paragraphs, avg_word_length, n_unique_words, lexical_diversity.
4. spaCy entities	20 min	Run nlp.pipe() on all 2,225 articles. Count PERSON / GPE+LOC / ORG.
5. Other features	10 min	n_caps_words, has_quote, first_word_length, headline_first_3_words.
6. Validate + save	5 min	Check schema. Check dtypes. Save .parquet.
7. Findings	10 min	Write findings.md.

Total: 90 minutes.

Phase C — Findings report for Olivia (10 minutes)

Write findings.md with these sections:

Final numbers

- Final row count: _____
- Articles per category: business , **entertainment** , politics , **sport** , tech _____
- Most common entity type overall: _____

Top 3 insights

1. _____
2. _____
3. _____

One question to investigate in Module 4

- _____ (Example: "Will adding TF-IDF on the body push accuracy past 95%?")

One chart that summarizes everything

Embed your most important chart (named entities by category).

Grading rubric (100 points)

Item	Points
All 17 columns exist with the right names and dtypes	25
Cleaning decisions written in markdown	10
Pipeline is reproducible (re-run from raw folders in ONE command)	15
spaCy named-entity counts work for all 3 types	10
All 2,225 articles loaded (no missing rows)	10
At least 3 exploratory + 1 explanatory chart per class	15
findings.md has insights, not just numbers	10
Code is clean (comments, sensible variable names)	5

Submit

Upload to module-3/class_6/submissions/<YourTeamName>/: - bbc_news_clean.parquet - Your Colab notebook (File > Download > Download .ipynb) - findings.md

Tips for the whole module

1. **Do NOT copy-paste code from this guide.** This guide gives you HINTS only. Write the code yourself. You will learn much faster.
2. **Save your notebook to Drive often.** Colab disconnects after 12 hours.
3. **Save intermediate .parquet files to Drive** (articles_step1, articles_step2, etc.). If something breaks, you do not redo everything.
4. **Pair-program.** One student types, one reads. Switch every 20 minutes.
5. **Make a chart BEFORE you write cleaning code.** You must SEE the problem before you can fix it.
6. **Make a chart AFTER each step.** Confirm your code did what you expected.
7. **Ask the mentor early.** If you are stuck for 20 minutes, ask. Do not waste an afternoon.
8. **Encouragement for first-time NLP teams:** Text is new for almost everybody. Take it slow. The dataset is small, so every experiment runs fast. Try things. If something breaks, the worst case is you wait 30 seconds for a re-run, not 30 minutes.
9. **Load spaCy ONCE.** This is the most common slow-down in Class 3. Load `nlp = spacy.load(...)` in ONE cell. Reuse it everywhere else.
10. **Print examples.** When you build a new feature, print 5 rows. "Is the first headline word really 'Real' for that Real Madrid article? Yes? Then the feature works."

Good luck.